



DEV305

laC 기반 EKS 멀티테넌시와 Sidecar-less 네트워킹

황우빈

데브옵스 엔지니어

네오위즈파트너스

이주안

솔루션 아키텍트

알고리스크퍼레이션



DEV305

멀티 클러스터를 위한 EKS Sidecar-less 네트워킹 전략

이주안

솔루션 아키텍트

알고리즘코퍼레이션

목차

- 알고리즘코퍼레이션 소개
- 우리가 Sidecar를 포기하게 된 이유
- Amazon VPC Lattice로 해결하는 다중 클러스터 연결
- Cilium으로 완성하는 통제
- 아키텍처 트레이드오프와 운영 가이드

알고리즘코퍼레이션



알고리즘코퍼레이션

비즈니스 혁신 여정을 함께하는 Cloud & AX 파트너

algorix

알고릭스코퍼레이션

algorix

2022



B2G EdTech 스타트업으로 시작
미국 K12 대상 코딩 교육 시스템 운영
California City SW Vendor

2024



AI 기술 R&D 및 AX 사업추진
행안부·국과수 딥페이크 AI 탐지기술 1위

2026



Pre-A 투자 유치
AI-Native 솔루션 개발

적은 DevOps 인력으로 EdTech 서비스를 운영해야 한다.

이상

- 서비스 규모 커져도 관리는 쉽게
- 정시 퇴근, 주말엔 알림 없이 휴식
- 서비스 추가? 배포하면 끝

현실

- 클러스터는 커지고 서비스는 늘어남
- Mesh 도입하려다 운영할게 2배
- 민감 정보라 외부에 못 맡김

우리가 Sidecar를 포기하게 된 이유

Multi-cluster가 되면 네트워크 문제는 달라집니다

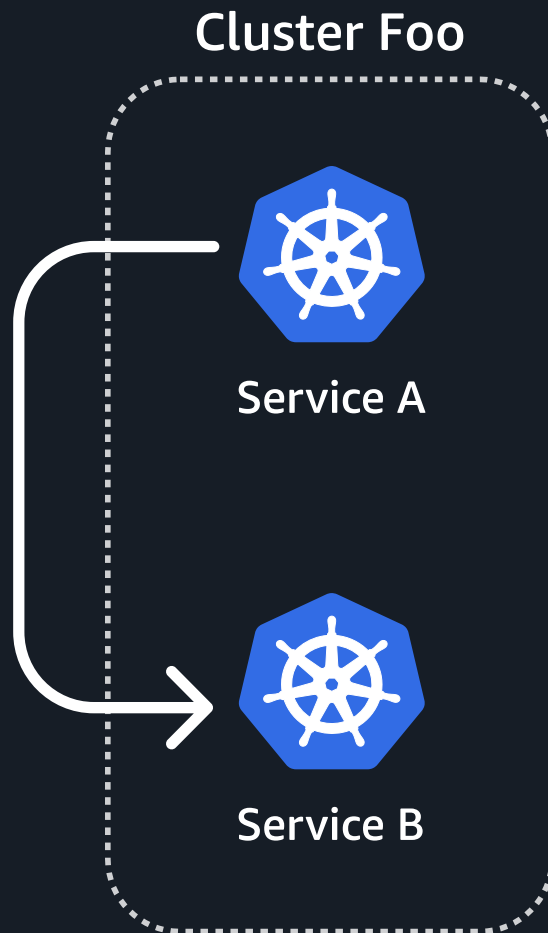
Single Cluster

- 연결과 통제가 한 곳에.
- 장애가 나도 볼 범위가 비교적 단순함

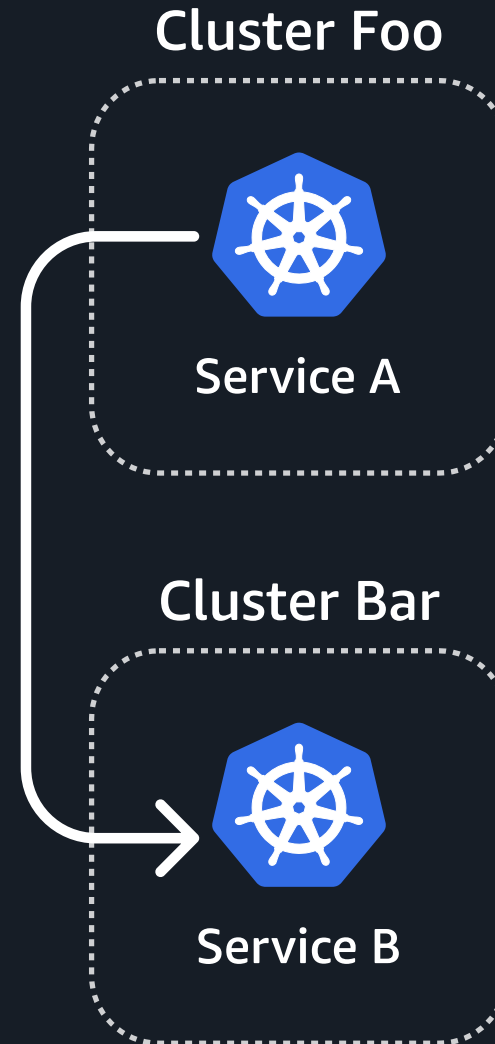
Multi Cluster

- 연결은 됐는데 왜 응답이 없지?
- 네트워크? DNS? Policy?
- 어떤 클러스터가 문제인지 파악하기 힘들

Single Cluster



Multi Cluster



가장 익숙한 선택: Service Mesh



CONNECTIVITY

연결

서비스 간 라우팅,
로드밸런싱



POLICY

정책

인증, 인가,
트래픽 제어



OBSERVABILITY

관측

메트릭, 로그,
트레이싱

그러나... Pod 수만큼 Envoy가 뜹니다.

우리가 겪은 문제점

강력했습니다. 그러나, 클러스터가 늘어나자 관리 범위도 같이 늘어났습니다.

Pod Lifecycle

Pod 라이프사이클 간섭

[시작] Envoy NotReady → 앱 연결 실패

[종료] Sidecar 먼저 종료 → 요청 유실

[배포] Injection 누락 → 관측 사각지대

Operations

운영 부담 누적

[OOM] 트래픽 급증 시 Sidecar OOM

[리소스] Pod 당 최소 +0.1vCPU, +128MiB

[디버깅] 장애 추적 Hop 증가 × 클러스터 수

서비스, 클러스터 규모가 커질수록 이 부담은 더 커집니다.

(예: 500 Pod 기준 → Envoy 전용 vCPU 50, Mem 64GB)



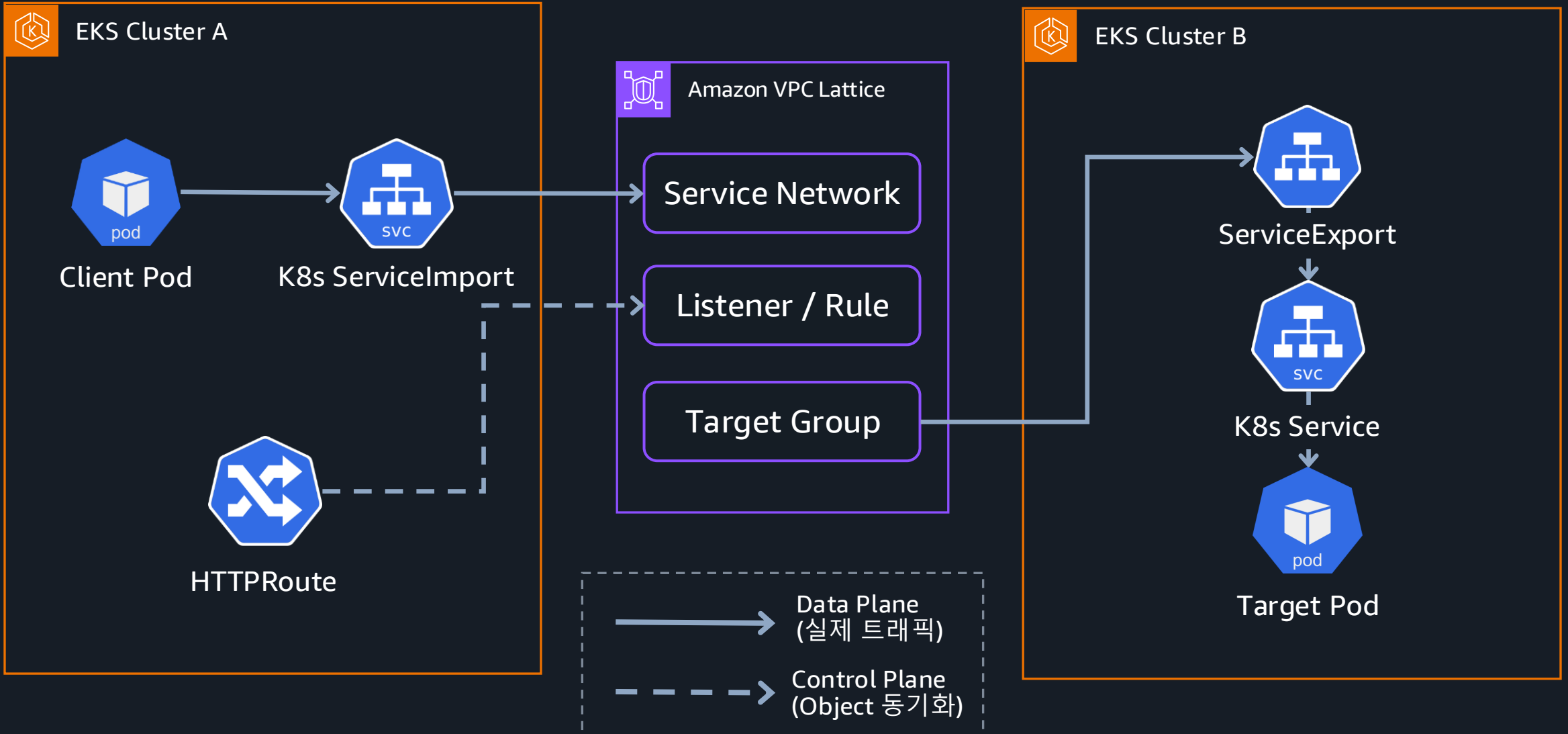
“그래서 우리는 연결은 인프라에 맡기고,
통제만 가져가기로 했습니다.”

Amazon VPC Lattice로 해결하는 다중 클러스터 연결

서비스 연결, 어떤 선택을 해야 할까요

	Istio Ambient Mesh	Cilium Cluster Mesh	Amazon VPC Lattice
연결 방식	East-West GW + mTLS 터널	Pod IP Direct 라우팅 or VXLAN Tunnel	Service Network 등록
VPC 제약	Peering/TGW 필요	Peering/TGW 필요 + CIDR 불가	제약 X, CIDR 겹쳐도 OK
운영 부담	Istiod, GW, ztunnel 관리	Agent, API server 관리	AWS Managed Controller
확장성	Mesh 확장 시 GW 추가 필요	터널 추가, 연결 구성 필요	Svc Network에 연결 추가
L7 트래픽 관리	Canary, Mirroring, Circuit Breaker	Gateway API (envoy 필요)	헤더/가중치 기반 라우팅

Gateway API로 보는 Lattice 연결



Lattice가 Sidecar 없이 동작하는 이유

1 Pod → 169.254.171.0/24 패킷 전송

2 Nitro Card가 H/W Level에서 인터셉트
(S/W 프록시 없음 → Sidecar 필요 없는 이유)

3 VPC Lattice Data Plane 처리

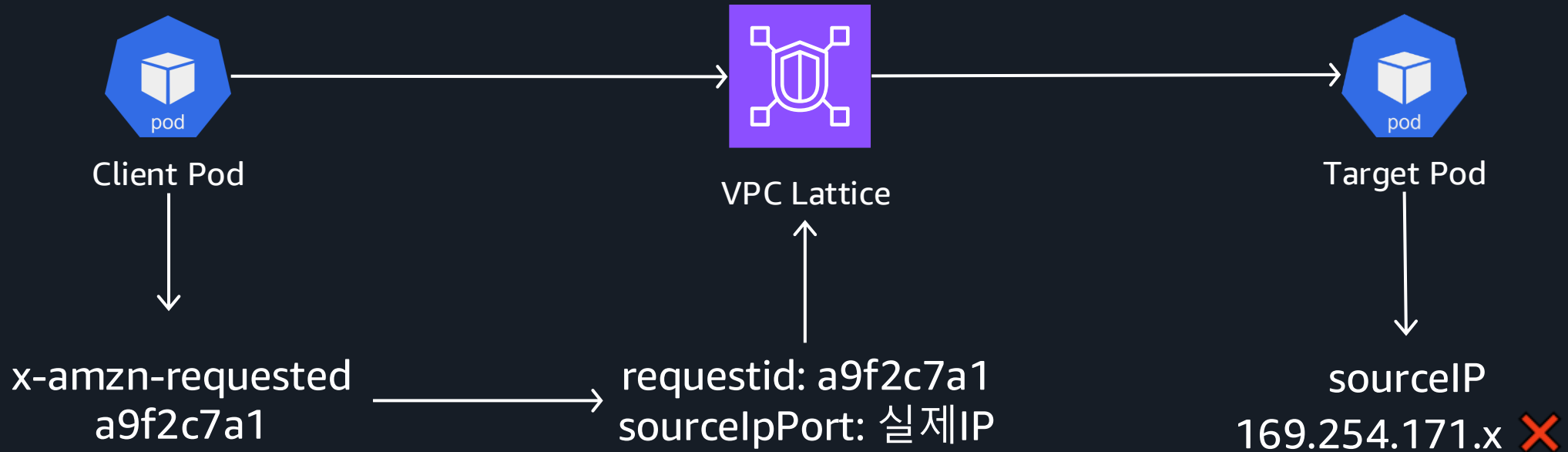
4 Target Pod 전달

⚠ 이때 Lattice가 **SNAT** 수행

Target Pod가 보는 Source IP = 169.254.171.x → 실제 Source Pod IP가 아님

Lattice에서 실제 Source는 누구일까

Target Pod가 보는 sourceIP = 169.254.171.0/24 (항상 SNAT)



requestid로만 **Source**를 역추적 가능

Port는 ServiceExport에서 exportedPorts로 정의

문제

HTTPRoute 사용 시,
Port 필드 지정해도 적용 X

```
HTTPRoute.yaml

backendRefs:
- name: payment-service
  kind: ServiceImport
  port: 8080 # 이 값, Lattice가 무시합니다
```

다른 Port로 트래픽 들어가거나,
연결이 안됨.

해결책

Port는 ServiceExport의
spec.exportedPorts에 명시해야 합니다.

```
ServiceExport.yaml

apiVersion: application-networking.k8s.aws/v1alpha1
kind: ServiceExport
metadata:
  name: payment-service
spec:
  exportedPorts:
  - port: 8080 # 여기
    routeType: HTTP
```

도입 시 주의사항

VPC Lattice 도입 시 짚고 넘어갈 부분

Policy

적용 범위

IAMAuthPolicy는
Gateway, Route에만 적용

직접 Service DNS 호출은
정책 적용 대상이 아님

Association

직접 연결의 한계

한 VPC는 Association으로
하나의 Service Network에만

여러 Network를 붙이려면
Endpoint 모델이 필요

Endpoint

IP 할당 제약

Service-network endpoint는
AZ별 연속 /28 IPv4 필요

Resource가 늘면
추가 /28이 더 필요할 수 있음

Cilium으로 완성하는 통제

우리 앞에 놓인 선택지

	AWS VPC CNI	Cilium	Calico (iptables)	Calico (eBPF)
Data Plane	iptables	eBPF	iptables	eBPF
NetworkPolicy 범위	미지원 (SG의존)	L3/L4/L7 + DNS/FQDN + HTTPRoute	L3/L4 + Global Policy	L3/L4 + Global Policy
Kube Proxy 대체	불가능	가능	불가능	가능
관측성	VPC Flow만	Hubble (L3-L7 Flow)	없음	없음
노드 간 암호화	없음	Wireguard + IPSec	Wireguard (선택)	Wireguard (선택)

EKS에 Cilium 도입하기

Option A

CNI Chaining

VPC CNI → IP 할당, Pod 연결

Cilium → 정책, 관측, LB

- 장점: 기존 VPC 네트워킹 유지
- 단점: 일부 고급 기능 제한

Option B



Cilium ENI 모드

VPC CNI, kube-proxy 제거

Cilium이 IPAM+정책+LB+관측 전담

- 장점: Cilium 전 기능 활용
- 주의: 노드 교체 필요

Zero-Downtime 마이그레이션 전략

1

기존 노드에 Label 설정

`kubectll labels node --all cni=aws-cni`

2

aws-node, kube-proxy를 Label Node에 고정

`nodeAffinity: cni=aws-cni`

3

Label 없는 새 노드 투입

Cilium DaemonSet만 실행

4

기존 Workload를 새 노드로 이동

`node drain` → Pod rescheduling

5

기존 노드 제거, aws-node DaemonSet 삭제

주의: Admission Webhook(Vault Injector, Karpenter 등)이 새 CIDR에서 접근 불가
→ `hostNetwork` 설정 필요

L3 – L7까지 Cilium으로 통제

L3/L4 + L7 정책 설정 예시

CiliumNetworkPolicy.yaml

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: api-allow-frontend
spec:
  endpointSelector: # 정책을 적용할 대상 Pod
    matchLabels:
      app: api-server
  ingress:
    - fromEndpoints: # 허용할 출발지 Pod
      - matchLabels:
          app: frontend
    toPorts: # L4: 포트 제한
      - ports:
          - port: "8080"
            protocol: TCP
      rules: # L7: HTTP 경로/메서드 제한
        http:
          - method: GET
            path: "/api/v1/.*"
```

1 Policy 적용할 Pod 지정

2 출발지 Pod 제한

3 L4 port + L7 path 제한

FQDN 기반 Egress Traffic 제어

DNS 기반 Egress 통제 예시

```
CiliumNetworkPolicy.yaml

apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-external-api-only
spec:
  endpointSelector: # 대상 Pod
    matchLabels:
      app: payment-service
  egress:
    - toEndpoints: # 클러스터 내 DNS 허용
      - matchLabels:
          k8s:io.kubernetes.pod.namespace: kube-system
          k8s-app: kube-dns
    toPorts:
      - ports:
          - port: "53"
            protocol: ANY
        rules:
          dns:
            - matchPattern: "*.algorix.services"
    - toFQDNs: # 특정 도메인만 외부 통신 허용
      - matchPattern: "*.algorix.services"
```

1

Policy 적용할 Pod 지정

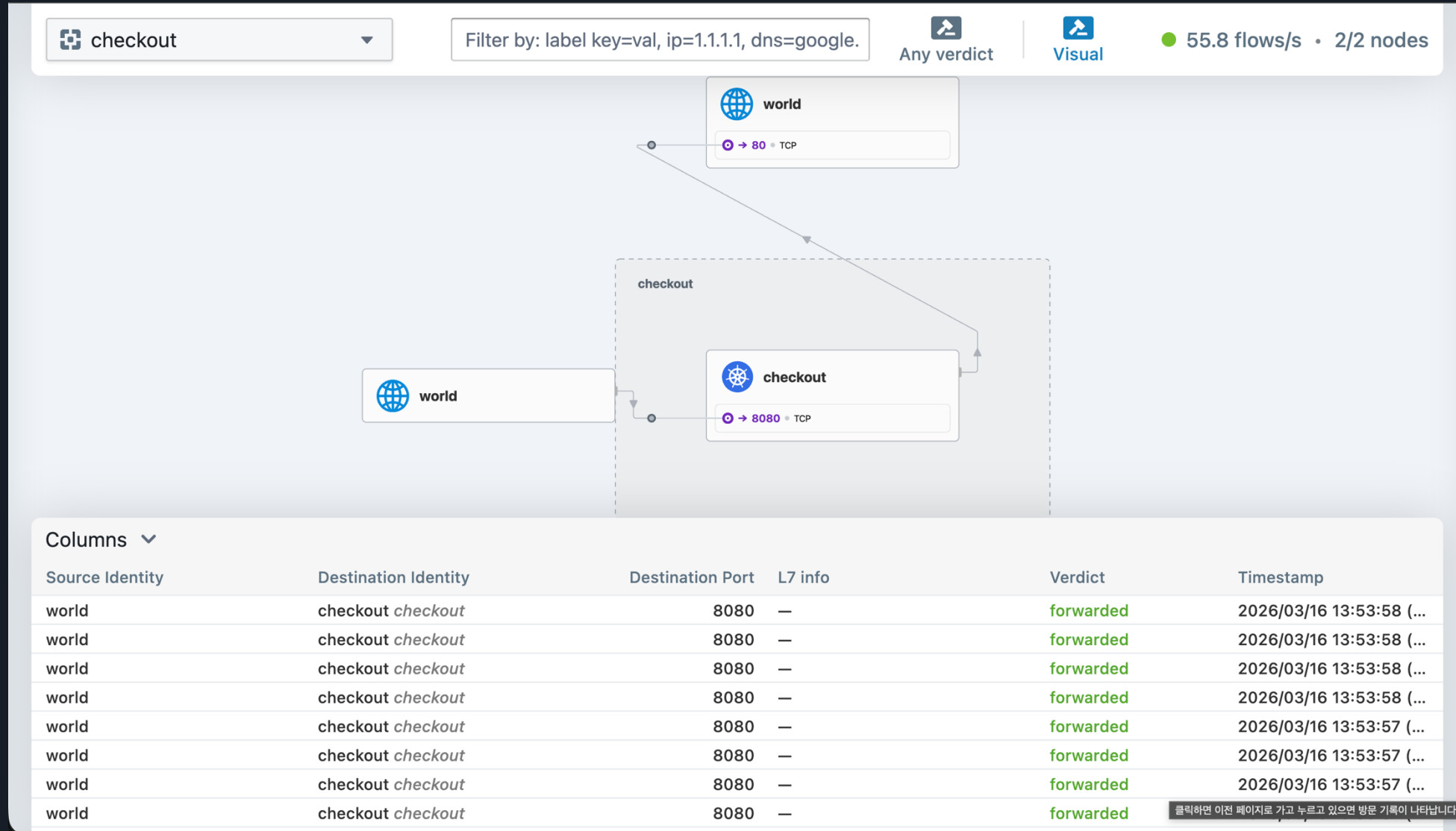
2

DNS 조회 허용 + FQDN 추적

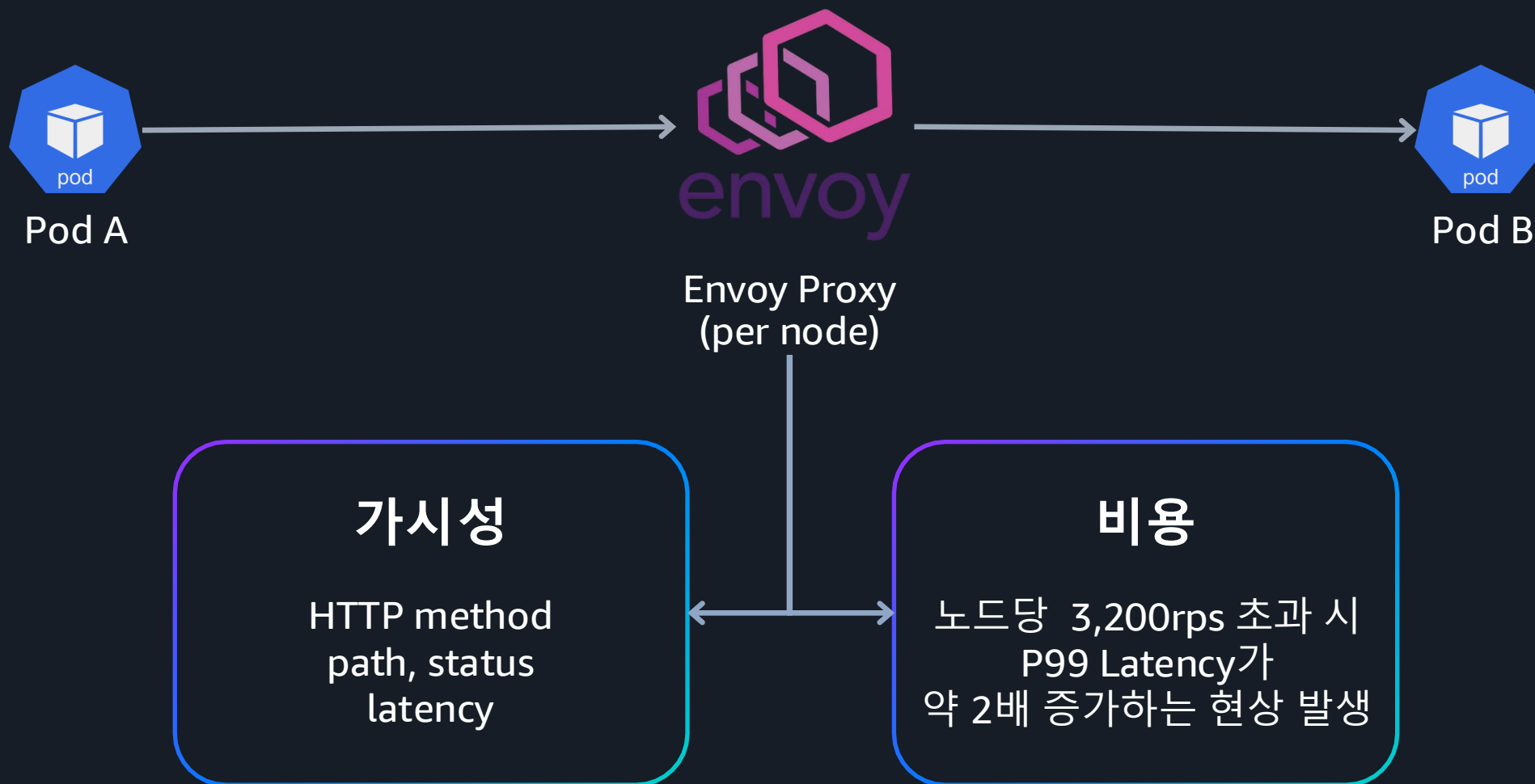
3

등록된 FQDN만 Egress 허용

Hubble로 모니터링하기



Hubble L7은 공짜가 아니다



Cilium + VPC Lattice 동시에 사용 시 생기는 문제

문제

Lattice DNS : 169.254.171.0/24
Link-local address.

bpf.masquerade = true
Pod → 169.254.171.x 트래픽 소실

해결책

Cilium ip-masq-agent 설정에서
Link-local을 명시적으로
non-masquerade에 포함

masqLinkLocal: false 시,
link local 대역이 자동 제외됨.

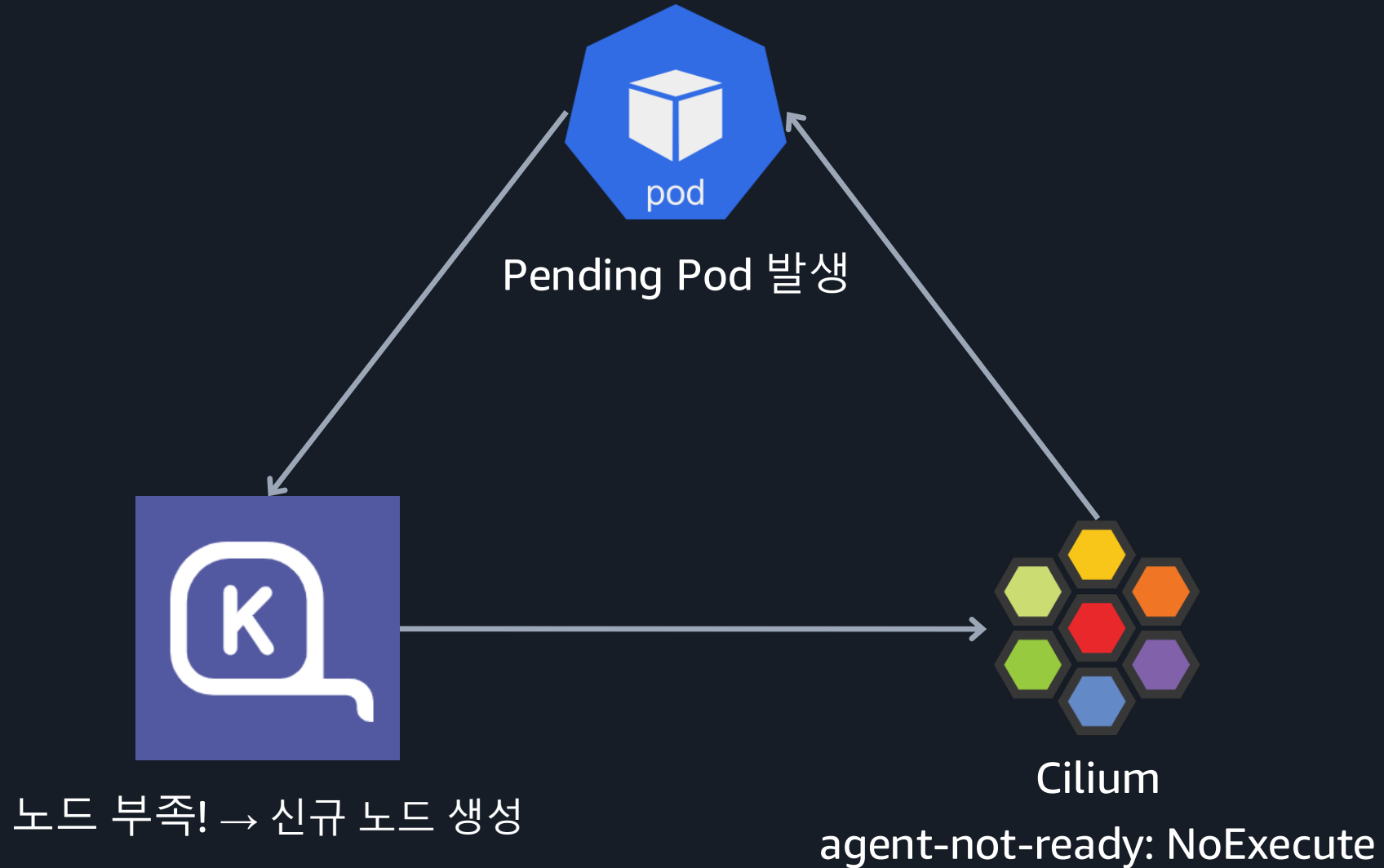
필요 시, nonMasqueradeCIDRs로 명시 가능

아키텍처 트레이드오프와 운영 가이드

아키텍처 설계 가이드

Lattice와 Cilium이 잘 맞는 상황	다른 선택을 고려해야 할 때
다중 클러스터 + 관리를 편하게 하고 싶을 때	Cross-Cluster mTLS를 전사적으로 강제해야 할 때
AWS에 인프라가 올인 된 환경일 때	다중 리전 아키텍처를 도입해야 할 때
Sidecar OOM이 서비스 장애로 이어진 경우	장시간 지속적인 연결이 필요할 때
Mesh 컨트롤 플레인을 직접 운영하고 싶지 않을 때	이미 다른 솔루션을 도입했고, 잘 동작하고 있는 경우.

Karpenter + Cilium 부트스트랩 루프



Karpenter + Cilium 부트스트랩 루프

1. Karpenter Deployment에 toleration 추가

```
karpenter-deploy-toleration.yaml

# karpenter Deployment
tolerations:
  - key: node.cilium.io/agent-not-ready
    operator: Exists
    effect: NoExecute
```

2. NodePool startupTaints 선언

```
karpenter-nodepool.yaml

# NodePool
spec:
  template:
    spec:
      startupTaints: # ← taints: 가 아님
        - key: node.cilium.io/agent-not-ready
          effect: NoExecute
```


마치며

1

Sidecar 제거를 통해 **300 Pod 기준**
76.8vCPU, 150GiB 리소스를 절감하였습니다.

2

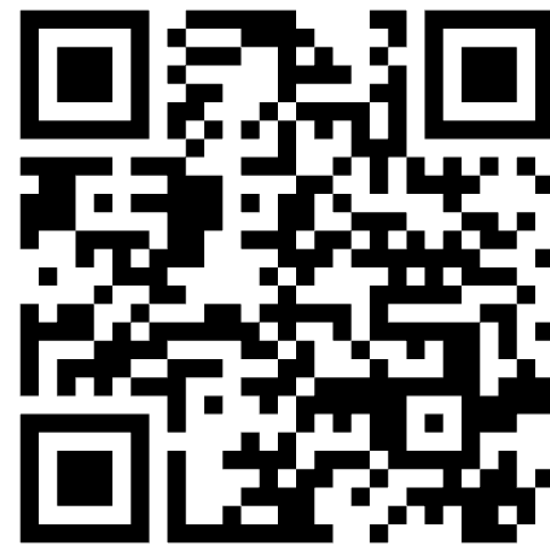
연결은 AWS에, 통제는 우리가.
이것이 Algorix의 해답이었습니다.

3

여러분의 클러스터에서는
그 선을 어디에 그을지 생각해보시기 바랍니다.



Thank you



Please complete the
session survey